

Rechnerarchitektur - Projekt - Aufgabe 1

Der in der Vorlesung vorgestellte Single-Cycle RISC-V Prozessor muss erweitert werden, damit er einen Bubble Sort Sortieralgorithmus unterstützt wird. Hierzu müssen die Insstruktionen auipc, bge und jalr implementiert werden. Erweitere dazu in Abbildung 2 für jeden dieser Befehle (falls nötig) den Datenpfad und füge weitere Kontrollsignale und Register hinzu. 1 Gib für jeden dieser Befehle auch den Wert aller Kontrollsignale an. Nimm dabei an, dass die ALU aus Abbildung 1 verwendet wird und gib don't cares explizit mit X an.

Die Abgabe soll ein Bild mit dem erweiterten Daten- und Kontrollpfad des Prozessors pro zusätzlichem Befehl (es können auch mehrere Erweiterungen in ein Bild gezeichnet werden) sowie eine Tabelle mit den Werten aller Kontrollsignale für die neuen Befehle beinhalten. Hinweis: Entnimm die Semantik der einzelnen RISC-V Befehle der in Moodle zur Verfügung gestellten RISC-V Reference Card

Aufbau des Prozessors vor den Änderungen

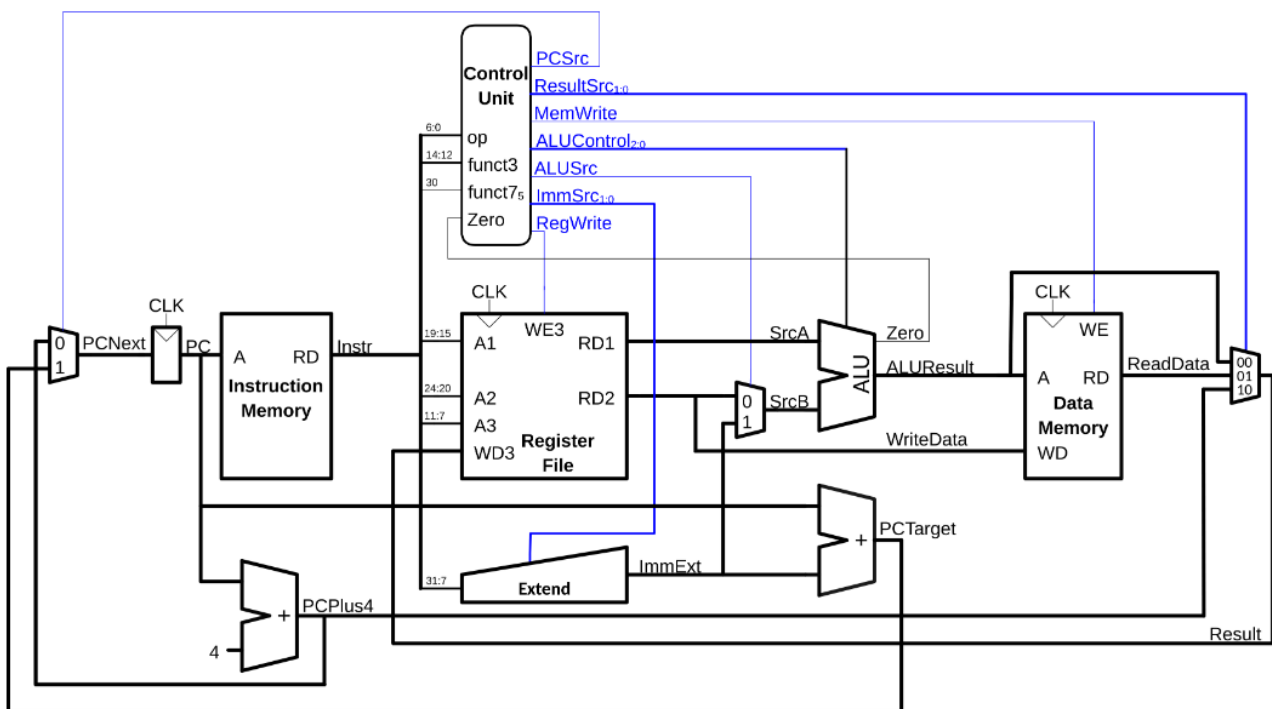


Abbildung 2: Schaltbild des Single-Cycle RISC-V Prozessors

Hinzufügen der Befehle

auipc

auipc	rd, upimm	add upper imm. to PC	U	0010111	-	-	rd = PC + (imm << 12)
-------	-----------	----------------------	---	---------	---	---	-----------------------

- Erweiterung von **Extend** für U-Type

```
when EXT_U_TYPE => immext <= instr(31 downto 12) & (11 downto 0 => '0');
```

- Im **PKG**, IMM_SRC_SIZE auf 3 Bit
 - Erweiterung ist notwendig, da aktuell nur 2 Bit somit sind nicht mehr zustände möglich
- Ergänzung in **Control Unit**

```
-- controls
  RegWrite & ImmSrc(2) & ImmSrc(1) & ImmSrc(0) & ALUSrc & MemWrite & ResultSrc(1:0) &
  Branch & ALUOp(1:0) & Jump & g_eCall;

-- signal controls
  STD_ULONGIC_VECTOR(12 downto 0); -- vorher 11 downto 0

-- Alle EXT_X_TYPE auf 3 Bits erweitern und U-Type definieren
  constant EXT_U_TYPE : std_ulogic_vector(IMM_SRC_SIZE-1 downto 0) := "100";

-- Zuweisung von controls when U-Type
  when "0010111" => controls <= "100100000000"; -- auipc
```

- **ALU Control** past bereits
- **MUX** der RD1, PC und op bekommt als Eingang

```
-- Wechsel zwischen PC und RD1
  pcOrRD1 <= PC when op = "0000111" else -- für AUIPC, PC bleibt die Basis
    RD1; -- für andere Instruktionen
```

Schaltung für auipc

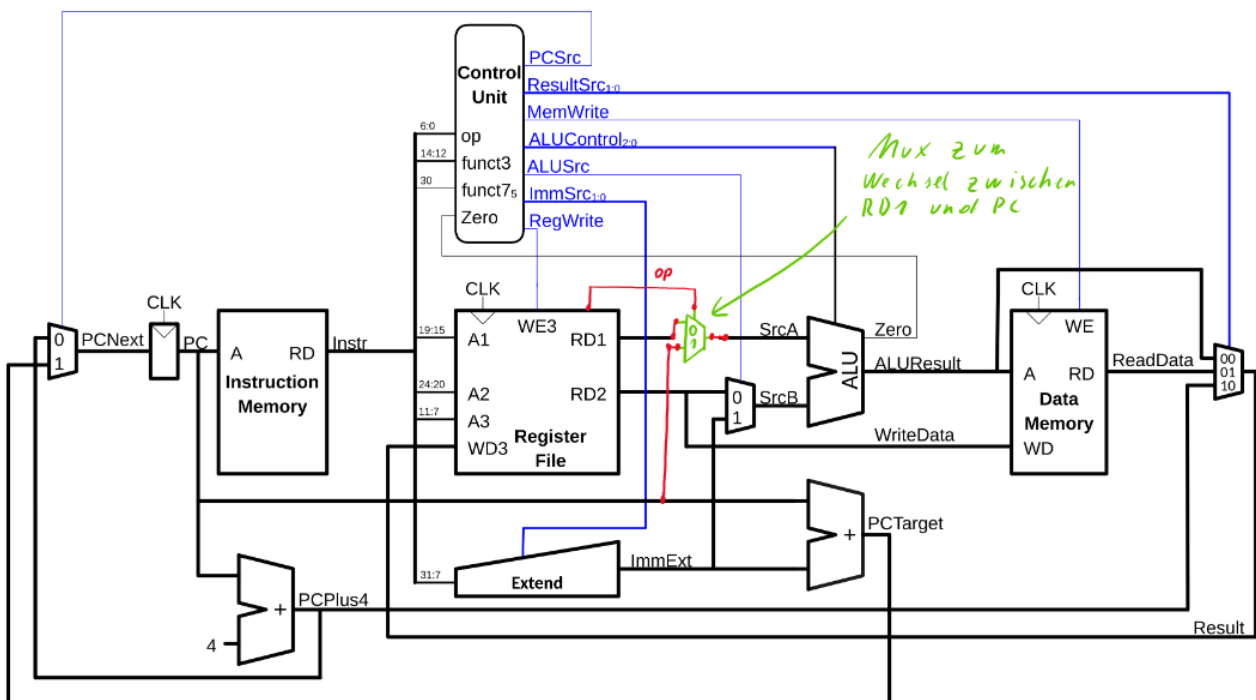


Abbildung 2: Schaltbild des Single-Cycle RISC-V Prozessors

bge

bge	rs1, rs2, label	branch $\geq$	B	1100011	101	-	if(rs1 $\geq$ rs2) PC += imm
-----	-----------------	---------------	---	---------	-----	---	------------------------------

- ALU

```
-- Neuen Ausgange sign hinzufügen
Sign: out STD_ULOGIC

-- sign auf 31 Bit setzen
Sign <= sum(31);
```

- Control Unit

```
-- Definieren von sign in der Alu Entity
Sign: in STD_ULOGIC;

-- Sign bit berücksichtigen
PCSrc <= (Branch and (
    (funct3 = "000" and Zero) or    -- beq
    (funct3 = "101" and not Sign) or -- bge
)) or Jump;
```

Schaltung für bge

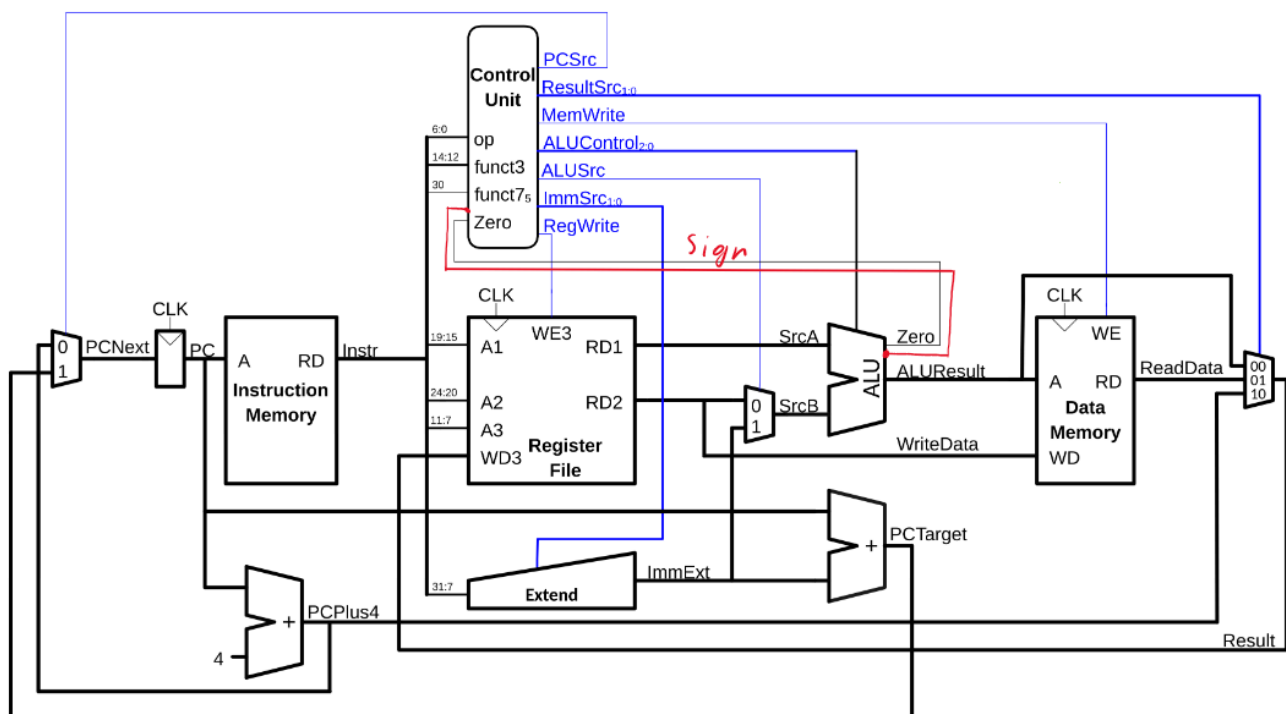


Abbildung 2: Schaltbild des Single-Cycle RISC-V Prozessors

jalr

jalr	rd, rs1, imm	jump and link reg	I	1100111	000	-	rd = PC+4; PC = rs1 + imm
------	--------------	-------------------	---	---------	-----	---	---------------------------

- Control Unit

```
-- jalr hinzufügen
when "1100111" => controls <= "111001000010";
```

- MUX vor PCTarget Adder

```
-- Wechsel zwischen RD1 und PC
pcOrRd1 <= PC when Jump = '0' else -- normaler Branch
RD1;                               -- jalr: rd1
```

- PC Target

```
-- Hochzählen von Pc ändern für jalr
y <= std_ulogic_vector(unsigned(pcOrRd1 ) + unsigned(ImmExt))
    and x"FFFFFFFE" when op = "1100111" -- maskieren
nur bei jalr
    else std_ulogic_vector(unsigned(pcOrRd1 ) + unsigned(ImmExt));
```

Schaltung für jalr

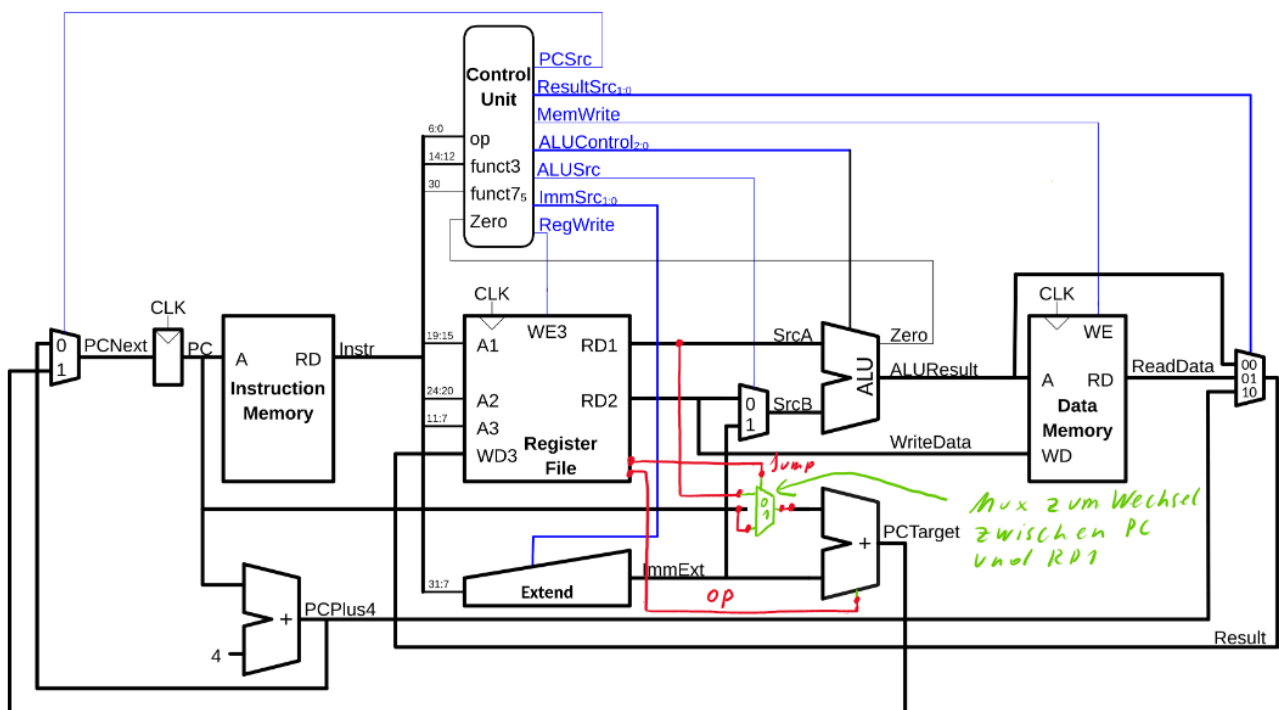


Abbildung 2: Schaltbild des Single-Cycle RISC-V Prozessors

Signale

Signal	auipc	bge	jalr
PCSrc	0	abhängig (Branch & -Sign)	1
ResultSrc	00	X	10
MemWrite	0	0	0
ALUControl	000 (ADD)	001 (SUB)	000 (ADD)
ALUSrc	1	0	1
ImmSrc	001 (U-Type)	100 (B-Type)	000 (I-Type)
RegWrite	1	0	1
Branch	0	1	0
Jump	0	0	1
SrcA	1 (PC)	0 (rd1)	0 (rd1)